

Lezione 2 - Organizzare progetti in C

ORGANIZZAZIONE CODICE

◆ Compilazione semplice

All'inizio lavoriamo con un unico file in cui tutto il codice è definito nello stesso posto.

```

#include <stdio.h>

struct stat
{
    int min;
    int max;
    float avg;
};

struct stat compute_stats(int *v, int len)
{
    struct stat s;
    s.min = v[0];
    s.max = v[0];
    s.avg = 0;

    for (int i = 0; i < len; i++)
    {
        if (v[i] < s.min) s.min = v[i];
        if (v[i] > s.max) s.max = v[i];
        s.avg += v[i];
    }

    s.avg /= len;
    return s;
}

int main()
{
    int v[8] = {2,3,5,7,-8,9,4,1};
    struct stat s = compute_stats(v, 8);
    printf("min:%d max:%d avg:%f\n", s.min, s.max, s.avg);
}

```

👉 Questo approccio funziona, ma tutto il codice è nello stesso file.

◆ Problema

A questo punto nasce un problema: il codice non è riutilizzabile.

Se vogliamo usare `compute_stats` in un altro programma, non possiamo copiarla ogni volta.

Serve quindi separare il codice in più file.

◆ Separazione in più file

Per rendere il codice riutilizzabile, lo dividiamo in più file.



Ad esempio:

- `main.c` → contiene il `main`
- `stat.c` → contiene la funzione

Nel `main.c` scriviamo:

```
#include <stdio.h>

struct stat
{
    int min;
    int max;
    float avg;
};

extern struct stat compute_stats(int * v, int len); // Funzione definita altrove

int main()
{
    int v[8] = {2,3,5,7,-8,9,4,1};
    struct stat s = compute_stats(v, 8);
    printf("min:%d max:%d avg:%f\n", s.min, s.max, s.avg);
}
```

👉 `extern` serve per dire al compilatore che la funzione esiste in un altro file.

Nel file `stat.c` abbiamo invece l'implementazione:

```

struct stat
{
    int min;
    int max;
    float avg;
};

struct stat compute_stats(int *v, int len)
{
    struct stat s;
    s.min = v[0];
    s.max = v[0];
    s.avg = 0;

    for (int i = 0; i < len; i++)
    {
        if (v[i] < s.min) s.min = v[i];
        if (v[i] > s.max) s.max = v[i];
        s.avg += v[i];
    }

    s.avg /= len;
    return s;
}

```

◆ Compilazione separata

Ora la compilazione avviene in più passi:

```

gcc -c main.c      # Crea main.o
gcc -c stat.c      # Crea stat.o
gcc -o main main.o stat.o

```

👉 prima si creano i file oggetto (.o), poi si fa il *linking*.

◆ Problema reale

A questo punto nasce un problema importante.

Abbiamo definito la stessa `struct stat` in più file.

Se anche solo cambia l'ordine dei campi, il programma continua a compilare ma produce risultati sbagliati.

```
struct stat
{
    float avg;
    int min;
    int max;
};
```

👉 la memoria viene interpretata in modo diverso → bug difficili da trovare

👉 Serve quindi un modo per definire queste strutture una sola volta e usarle ovunque. Questo porta all'introduzione dei file header (.h).

◆ File header (.h)

Per evitare duplicazioni e problemi con le `struct`, si introducono i file header (`.h`).

L'idea è semplice:

- definiamo **struct e dichiarazioni una sola volta**
- le includiamo dove servono



Esempio di header

```
#ifndef _STAT_H
#define _STAT_H

struct stat
{
    int min;
    int max;
    float avg;
};

struct stat compute_stats(int * v, int len); // Dichiarazione funzione

#endif
```

👉 questo file contiene:

- definizione della struct
- dichiarazione della funzione
- protezione da inclusioni multiple (*include guard*)

👉 evita che lo stesso header venga incluso più volte causando errori

◆ Uso degli header

A questo punto i file `.c` diventano più puliti.

stat.c

```
#include "stat.h"

struct stat compute_stats(int *v, int len)
{
    struct stat s;
    s.min = v[0];
    s.max = v[0];
    s.avg = 0;

    for (int i = 0; i < len; i++)
    {
        if (v[i] < s.min) s.min = v[i];
        if (v[i] > s.max) s.max = v[i];
        s.avg += v[i];
    }

    s.avg /= len;
    return s;
}
```

👉 qui implementiamo la funzione dichiarata nell'header.

main.c

```
#include <stdio.h>
#include "stat.h"

int main()
{
    int v[8] = {2,3,5,7,-8,9,4,1};
    struct stat s = compute_stats(v, 8);
    printf("min:%d max:%d avg:%f\n", s.min, s.max, s.avg);
}
```

👉 non serve più `extern`, perché la funzione è già dichiarata nell'header.

◆ Vantaggio

Ora:

- la `struct` è definita una sola volta
- non ci sono duplicazioni
- il codice è più organizzato

👉 separiamo:

- **interfaccia (.h)**
- **implementazione (.c)**

A questo punto possiamo estendere il concetto: non solo separare file, ma riutilizzare codice tra programmi diversi.

📌 LIBRERIE

◆ Librerie statiche

Quando vogliamo riutilizzare codice in più programmi, possiamo raccoglierlo in una **libreria statica** (`.a`).

L'idea è:

- compiliamo i file `.c` → otteniamo `.o`
- li uniamo in una libreria
- poi colleghiamo la libreria al programma



Creazione libreria

```
gcc -c stat.c           # Crea stat.o
ar r libstat.a stat.o   # Crea la libreria
ranlib libstat.a        # Crea l'indice (facilita l'uso)
```

👉 la libreria statica è un archivio di file oggetto (`.o`)

Durante il linking, il codice necessario viene copiato dentro l'eseguibile.

👉 l'eseguibile finale sarà più grande, perché contiene il codice della libreria



Uso della libreria

```
gcc -L. -o main main.c -lstat
```

👉 opzioni importanti:

- `-L.` → indica dove cercare la libreria
- `-lstat` → usa `libstat.a` (si omettono "lib" e ".a")

◆ Librerie dinamiche

Le librerie dinamiche (`.so` su Linux, `.dylib` su macOS, `.dll` su Windows) vengono caricate **a runtime**, cioè quando il programma viene eseguito.

Questo permette di:

- aggiornare la libreria senza ricompilare tutto
- condividere la stessa libreria tra più programmi
- ridurre la dimensione dell'eseguibile



Creazione libreria dinamica

Per creare una libreria dinamica, il codice deve essere indipendente dalla posizione in memoria.

Per questo si usa l'opzione:

```
gcc -fPIC -c stat.c  
gcc -shared -o libstat.so stat.o
```

👉 `-fPIC` → codice indipendente dalla posizione

👉 `-shared` → crea la libreria dinamica



Linking

```
gcc -L. -o main main.c -lstat
```

👉 -L. → directory dove cercare la libreria

👉 -lstat → usa libstat.so

◆ Esecuzione

A runtime, il sistema deve sapere dove trovare la libreria.

Su Linux:

```
export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH
```

Su macOS:

```
export DYLD_LIBRARY_PATH=.:$DYLD_LIBRARY_PATH
```

👉 se la libreria non viene trovata, il programma non parte

◆ Verifica

Possiamo controllare le librerie usate da un programma:

- Linux → `ldd main`
- macOS → `otool -L main`

◆ Differenza con statiche

- **statica** → il codice viene copiato nell'eseguibile
- **dinamica** → il codice viene caricato a runtime

◆ Idea

Le librerie servono per:

- riutilizzare codice
- organizzare progetti grandi
- separare sviluppo e utilizzo

MAKE

◆ Perché serve make

Quando il progetto cresce e abbiamo molti file `.c` e `.h`, gestire manualmente le dipendenze diventa complicato.

Ogni volta che modifichiamo un file:

- dovremmo ricompilare solo quello
- poi rifare il linking

👉 non ha senso ricompilare tutto ogni volta

Serve quindi un modo per automatizzare questo processo.

◆ Idea di make

Con *make* definiamo delle regole:

- **target** → cosa vogliamo ottenere
- **prerequisiti** → da cosa dipende
- **comando** → come costruirlo

◆ Esempio logico

Vogliamo costruire `main` a partire dai file oggetto:

```
gcc -o main main.o stat.o
```

👉 quindi:

- target → main
- prerequisiti → main.o , stat.o

◆ Makefile base

Un Makefile è un file con regole nel formato:

```
target: prerequisiti
        comando
```

⚠ il comando deve iniziare con **TAB**, non spazi



Esempio completo

```
all: main
```

```
main: main.o stat.o
        gcc -o main main.o stat.o
```

```
stat.o: stat.c stat.h
        gcc -c stat.c
```

```
main.o: main.c stat.h
        gcc -c main.c
```

👉 questo Makefile:

- costruisce main
- ricompila solo ciò che è cambiato

◆ Come funziona

Quando eseguiamo:

make

make:

- controlla le dipendenze
- ricompila solo i file modificati
- evita lavoro inutile

👉 esempio:

- modifichi `stat.c` → ricompila solo `stat.o` + linking
- non modifichi nulla → non fa nulla

◆ Target utili

clean

```
clean:
    rm *.o
```

👉 elimina i file oggetto

.PHONY

```
.PHONY: clean
```

👉 indica che `clean` non è un file reale

◆ Variabili

Per evitare ripetizioni:

```
CC = gcc
```

```
CFLAGS = -O3
```

```
main:
```

```
$(CC) $(CFLAGS) -o main main.o stat.o
```

◆ Regole generiche

```
%.o: %.c
```

```
gcc -c $<
```

👉 \$< indica il primo prerequisito (cioè il file .c)

👉 crea automaticamente i .o dai .c

◆ Funzionalità avanzate di make

Make mette a disposizione strumenti più avanzati per gestire progetti complessi.

Wildcard

Possiamo selezionare gruppi di file automaticamente:

```
SRC = $(wildcard *.c)
```

👉 seleziona tutti i file .c nella directory

Trasformazione nomi (template)

Possiamo trasformare una lista di file:

```
OBJ = $(SRC:.c=.o)
```

👉 converte automaticamente i .c in .o

Esecuzione condizionale

Make permette anche condizioni:

```
ifeq ($(CC), gcc)
    CFLAGS = -O3
endif
```

👉 esegue codice solo se la condizione è vera

Altri sistemi di build

Per progetti più grandi, make può diventare difficile da gestire.

Esistono altri strumenti:

- **CMake** → genera automaticamente Makefile
- altri sistemi di build più avanzati

👉 vengono usati per progetti complessi e multiplatforma

◆ Idea fondamentale

make serve per:

- automatizzare la compilazione
- ricompilare solo ciò che serve
- gestire progetti grandi

COMPILATORI

◆ Opzioni comuni del compilatore

Il compilatore mette a disposizione diverse opzioni utili per controllare il comportamento della compilazione.

Le principali categorie sono:

- **ottimizzazione** → migliorare le prestazioni (-O0 , -O1 , -O2 , -O3)
- **debug** → includere informazioni per il debug (-g)
- **warning** → segnalare possibili problemi (-Wall , -Wextra , -pedantic)
- **standard** → specificare la versione del linguaggio (-std=c99 , -std=c11 , ecc.)

👉 queste opzioni possono essere combinate tra loro durante la compilazione

◆ Ottimizzazione del compilatore

Il compilatore può applicare diverse ottimizzazioni per migliorare le prestazioni del codice.

I livelli principali sono:

- -O0 → nessuna ottimizzazione
- -O1
- -O2
- -O3 → massimo livello (in genere)



Esempio

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    srand(time(NULL));

    int n = 1<<20;                // dimensione vettore
    int *v = (int *) malloc(n * sizeof(int));

    for (int i = 0; i < n; i++) {
        v[i] = rand() % 10;        // inizializzazione casuale
    }

    int dot = 0;

    clock_t start = clock();

    for (int i = 0; i < n; i++) {
        dot += v[i] * v[i];        // prodotto interno
    }

    clock_t end = clock();

    float ms = (end - start) / (CLOCKS_PER_SEC/1000.0);

    printf("dot = %d\n", dot);
    printf("tempo = %f ms\n", ms);
}
```

👉 stesso codice, ma tempi diversi a seconda delle ottimizzazioni.



Compilazione

```
gcc -O0 -o main0 main.c
gcc -O1 -o main1 main.c
gcc -O2 -o main2 main.c
gcc -O3 -o main3 main.c
gcc -O3 -march=native -o mainN main.c
```

👉 `-march=native` sfrutta le caratteristiche della CPU

◆ Debug

Per il debug si usa:

```
-g
```

👉 mantiene informazioni utili per strumenti come `gdb`

◆ Warning del compilatore

Il compilatore può segnalare problemi nel codice.

Opzioni principali:

- `-Wall` → warning base
- `-Wextra` → warning aggiuntivi
- `-pedantic` → rispetto dello standard



Esempio

```
#include <stdio.h>
#include <stdlib.h>

// codice scritto per generare warning

int f(int x, int y)
{
    return x*x;                // y non usato
}

int main()
{
    short a[3] = {1,2,3};
    int b = 1<<64;             // overflow

    printf("%d\n", b);

    for (int i = 0; i < 3; i++) {
        printf("%d %d\t", a[i], f(a[i], 1));
    }

    int *p = malloc(sizeof(int)); // memoria non liberata
    printf("%p\n", p);
}
```



Esempi di problemi:

- overflow ($1 \ll 64$)
- variabili non usate
- gestione memoria non corretta

◆ Standard del C

Possiamo specificare la versione del linguaggio:

- -std=c89
- -std=c99

- `-std=c11`
- `-std=c17`

👉 utile per garantire compatibilità

◆ Idea finale

Le opzioni del compilatore servono per:

- migliorare le performance (`-O`)
- fare debug (`-g`)
- individuare errori (`-Wall` , `-Wextra`)
- controllare lo standard (`-std`)